# 基于灰箱模型与机器学习回归算法的巨大肾积水输尿管流量-压降关系研究


## 摘要

随着微创泌尿外科和精准医学的快速发展，泌尿系统梗阻性疾病的病理机制研究和手术治疗优化已成为临床医学与人工智能领域交叉研究的重要课题。本文以巨大肾积水这一典型泌尿系统梗阻疾病为研究对象，通过数学建模与机器学习算法对比分析，系统研究了输尿管流量及其与病理指标之间的关系。研究结果为临床诊断提供了定量化分析工具，为术后疗效的动态监测和术后评估提供了科学参考，对辅助泌尿外科诊疗和术后疗效评估具有一定的临床应用价值。

**针对问题一，**基于**蠕动泵送理论**和含修正系数 $K_2$的**泊肃叶定律**，通过修正项描述流固耦合效果，简化得到流量-压降的线性模型，结合**约束最小二乘拟合**确定无量纲常量 $K_1=1$，修正项 $K_2=0.0908$，建立了输尿管流量-压降模型 $Q=f\cdot A\cdot D^2+\frac{0.0908(\pi D^4\cdot \Delta p)}{128\mu\cdot L}$，模型决定系数 **$R^2=0.9465$**，拟合效果极佳。

**针对问题二**，利用正常肾脏输尿管流量数据，基于支持向量回归( SVR) 、轻量级梯度提升( LightGBM) 、Lasso 回归、偏最小二乘回归（PLS）四种机器学习算法分别构建了输尿管流量预测模型，基于**网格搜索**的参数调优方法对所有的超参数组合进行模型调优。其中 **Lasso** 模型测试集在各项评估指标中均展现良好，表明其最能够有效显现输尿管流量与压降等生理指标之间的复杂关系。

**针对问题三**，通过解析式模型与 AI 模型对手术前后输尿管流量预测结果与临床实测数据的对比，发现在理论解释性与预测精度方面，基于**流体力学**原理构建的解析模型相对**误差小于 1%**，但认为仍可通过**增加机理分析**减少数据拟合提高模型泛用性与可解释性。机器学习模型由于数据量较少且病人数据超出训练范围，预测表现一般，认为应**增加训练数据量**并增大训练数据范围来优化模型预测能力。



**关键词**：巨大肾积水；约束最小二乘拟合；灰箱模型；机器学习；Lasso 回归

# 一、问题重述

## 1.1 问题背景

巨大肾积水是一种以肾盂和肾盏显著扩张为特征的泌尿系统疾病，在成人中相对少见，常见于婴幼儿和儿童中，主要由肾盂输尿管连接处狭窄引起。该疾病可导致肾实质受压萎缩，严重影响肾功能，若不及时干预，可能造成不可逆的肾脏损伤。本题中，医院创新性地提出了一种新型手术方案：在传统肾盂减压引流的基础上，进一步实施肾脏整形修复术。具体包括：1）修剪扩张的肾盂组织；2）重塑肾脏形态，使其恢复接近正常的椭球结构；3）采用腹腔内网兜悬吊技术固定肾脏，以促进其解剖位置和功能的逐步恢复。术后输尿管尿流量的恢复情况是评估手术成功的关键指标之一，直接影响患者的长期预后。因此，建立精确的输尿管流量-压降预测模型，对于术前评估、手术方案优化以及术后疗效监测均具有重要意义。

## 1.2 问题提出

根据临床检测数据和患者病历资料，其中附件 1 为正常肾脏某一患者手术前后的输尿管流量数据，附件 2 为术前术后诊断结果。基于附件数据建立数学模型，解决以下三个问题：

针对问题一：利用附件 1 中的正常肾脏输尿管流量数据，建立输尿管内流量 Q 与压降ΔP 的数学模型。要求模型需综合考虑：1）输尿管蠕动特征（频率 f 和幅度 A）；2）输尿管几何参数（管径 D 和长度 L）；3）尿液粘度μ等参数。模型应能同时表征输尿管蠕动的主动泵送效应和压降对尿流的阻力效应。

针对问题二：结合人工智能技术，使用附件 1 中的正常肾脏输尿管流量数据，构建基于 AI 算法的输尿管流量-压降数学模型。

针对问题三：基于附件 1 提供的手术前后数据，分别应用前两问建立的解析模型和 AI 模型预测输尿管泌尿量。通过与附件 2 中的实际临床数据进行对比分析，评估模型预测效果，讨论模型局限性，并提出针对性的优化改进方案。

# 二、问题分析

针对问题一，通过机理分析和数据拟合两种方式构建基于灰箱模型的流量-压降模型，机理分析方面，认为总流量等于蠕动引发的流量和肾盂膀胱压力差引发的流量之和，蠕动引发的流量由蠕动泵送理论表示（描述了蠕动的主动泵送效应），压降引发的流量由泊肃叶定理表示（描述了压降对流量的阻力效应），得到的模型满足条件 1）包含各种参数，由于输尿管非刚体圆柱管道，所以引入修正系数 $k_2$。数据拟合方面利用最小二乘确定无量纲常数 $k_1$、修正系数 $k_2$得到压降流量模型，并通过 $R^2$，RMSE 评估模型对数据拟合程度。

针对问题二，本文采用多算法对比的研究方法，并引入网格搜索方法对超参数模型进行调优，进一步提高结果准确性。基于 50 例临床数据，本文系统评估了支持向量回归( SVR) 、随机森林、轻量级梯度提升( LightGBM) 、Lasso 回

归四种机器学习算法的预测性能及结果合理性，并基于网格搜索参数调优方法选择最优模型。

针对问题三，对比预测的病人数据和真实数据，进行误差分析，根据误差分析讨论局限性，解析式模型局限性来源为机理分析时进行的假设和参数拟合时的误差，AI 模型局限性来源于数据数量和范围，最后根据局限性提出相应的优化建议来提高模型泛用性。

# 三、模型假设与符号说明

## 3.1 模型基本假设

本文提出以下合理假设：

（1）输尿管为均匀刚性管道；
（2）尿液为牛顿流体，粘度恒定；
（3）蠕动幅度和蠕动频率均维持稳定；
（4）忽略重力因素；
（5）假设表中所测数据均为可信。
（6）假设输尿管流量等于泌尿量等于 GFR。

## 3.2 符号说明

**表 1 符号说明**

| 符号 | 意义 | 单位 |
|---|---|---|
| $Q$ | 输尿管内流量 | $m^3/s$ |
| $\Delta p$ | 压降 | Pa |
| $f$ | 蠕动频率 | Hz |
| $A$ | 蠕动幅度 | m |
| $D$ | 输尿管管径 | m |
| $L$ | 输尿管长度 | m |
| $\mu$ | 尿液粘度 | Pa·s |
| $K_1$ | 无量纲常量 | / |
| $K_2$ | 引入泊肃叶定理修正系数 | / |

# 四、数据预处理

## 4.1 指标选取

选取输尿管蠕动频率、幅度、管径、长度、压降及尿液粘度作为输入变量。

## 4.2 数据统计

经统计认为，数据无异常值，无缺失值。附件 1 中流量平均值为 8.85e-07 ，标准差为 1.59e-07，最小值为 6.17e-07 ，最大值为 1.17e-06 ，中位数为 8.75e-07，25%和 75%处分别对应 7.54e-07 和 9.96e-07，单位（$m^3/s$），数据较为集中，说明样本在此方面的生理特征差距较小。

**表 2 流量及相关影响因素描述性统计分析结果**

| 指标 | 平均值 | 标准差 | 最大值 | 最小值 | 峰度 | 偏度 | 变异系数 |
|---|---|---|---|---|---|---|---|
| 蠕动频率(Hz) | 0.148 | 0.022 | 0.182 | 0.108 | -0.68 | 0.15 | 0.15 |
| 蠕动幅度(m) | 0.00103 | 0.00018 | 0.00135 | 0.00072 | -0.32 | 0.21 | 0.17 |
| 管径 (m) | 0.0045 | 0.00054 | 0.0054 | 0.0037 | -0.41 | -0.02 | 0.12 |
| 长度 (m) | 0.265 | 0.029 | 0.32 | 0.21 | -0.53 | 0.18 | 0.11 |
| 粘度 (Pa · s) | 0.00102 | 0.00009 | 0.0012 | 0.00089 | -0.27 | 0.32 | 0.09 |
| 压降 (Pa) | 253.76 | 45.82 | 373.10 | 184.18 | 0.41 | 0.52 | 0.18 |
| 流量 ($m^3/s$) | 8.85e-07 | 1.59e-07 | 1.17e-06 | 6.17e-07 | -0.52 | 0.18 | 0.18 |
| 流量(mL/min) | 52.06 | 10.65 | 70.00 | 37.00 | -0.59 | 0.16 | 0.20 |

对稳定性的分析：粘度（变异系数 0.09）＞长度（0.11）＞管径（0.12）＞蠕动频率（0.15）＞蠕动幅度（0.17）＞流量（0.18-0.20），可见流体自身属性（粘度）和几何参数（长度、管径）的稳定性最高。

对分布特征的分析：绝大部分指标的峰度均为负值（平顶峰度），偏度绝对值均＜0.3（近似对称），数据分布均匀，无显著极端值（异常值风险低），说明样本数据质量较高，适合进一步分析各因素与流量的相关性。

# 五、模型建立与求解

## 5.1 问题一模型建立与求解

建立流量-压降预测模型，具体建模过程如下：

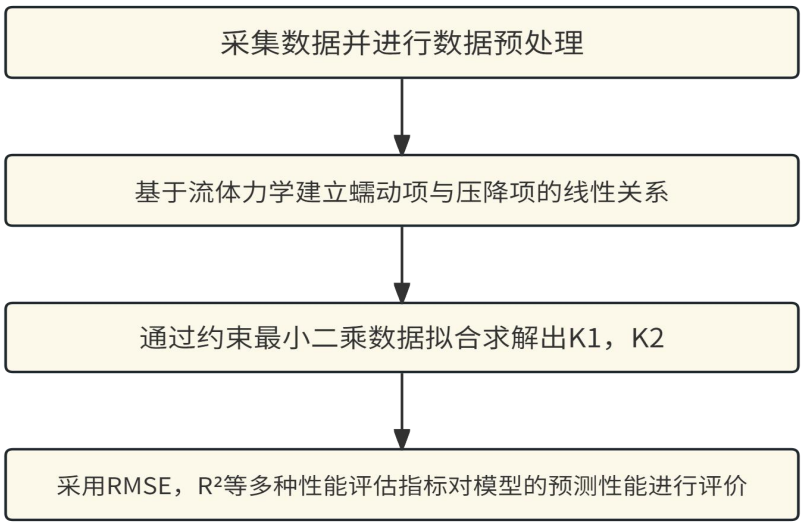



**图 2 第一问求解流程图**

### 5.1.1 求解思路

结合蠕动泵送理论和泊肃叶定律，建立流量关于无量纲常数 $K_1$和修正量 $K_2$ 的综合模型[1]，再通过附件一的样本数据进行拟合，获取最合适的 $K_1$和 $K_2$值。

### 5.1.2 模型建立

总流量等于蠕动引发的流量加压降引发的流量

$$Q=Q_{蠕动}+Q_{压降} \tag{1}$$

蠕动引发的流量由蠕动泵送理论公式描述,该式体现了蠕动的主动泵送效应

$$Q_{蠕动}=K_1\cdot f\cdot A\cdot D^2 \tag{2}$$

压降引发的流量由含修正量 $k_2$的泊肃叶定理描述，假设 $k_2$可以修正输尿管非圆柱刚体的缺陷，泊肃叶定理可以描述管对液体的阻滞作用，该式体现了压降对流量的阻碍作用，阻碍作用由管与液体属性决定。

$$Q_{压降}=\frac{K_2\pi D^4\Delta p}{128\mu\cdot L} \tag{3}$$

最终建立流量-压降综合模型

$$Q=K_1\cdot f\cdot A\cdot D^2+\frac{K_2\pi D^4\cdot\Delta p}{128\mu\cdot L} \tag{4}$$

5.1.3 求解与分析

通过最小二乘拟合附件 1 数据，确定 $K_1$，$K_2$最佳值，最后分析模型合理性。

step1 构建特征项

基于流体力学理论构建了两个关键特征项

蠕动项

$$term1 = f \cdot A \cdot D^2 \tag{5}$$

压降项

$$term2 = \frac{\pi D^4 \cdot \Delta p}{128 \mu \cdot L} \tag{6}$$

step2 线性模型建立

$$Q_{actual} = K_1 \cdot term1 + K_2 \cdot term2 \tag{7}$$

其中 $k_1$和 $k_2$为待求解的模型参数

step3 约束最小二乘参数估计

（1）设计矩阵 X = [term1, term2]

（2）设置参数约束：$K_1 > 0$（物理意义要求蠕动项系数为正）

（3）MATLAB 中使用 lsqlin 函数，采用内点算法 （interior-point）求解

（4）得到最优参数 $K_1$=1 和 $K_2$=0.0908

step4 模型评估与可视化

模型决定 $R^2$=0.9465，均方根误差 RMSE=3.63e-8，拟合效果极佳，认为模型建立成功。

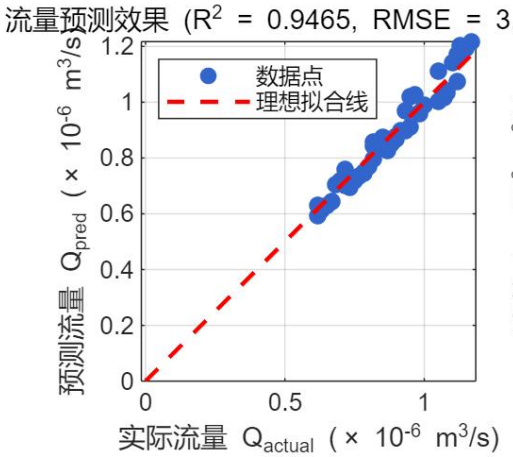



图 3 流量预测效果图

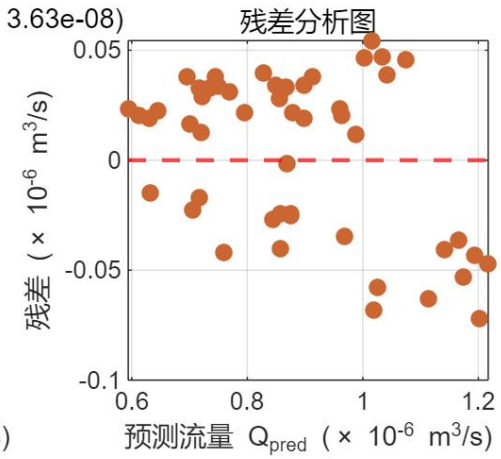



图 4 残差分析图

可视化预测值与实际值并对比，可以看出绝大部分预测较好，再次说明模型的良好性能。

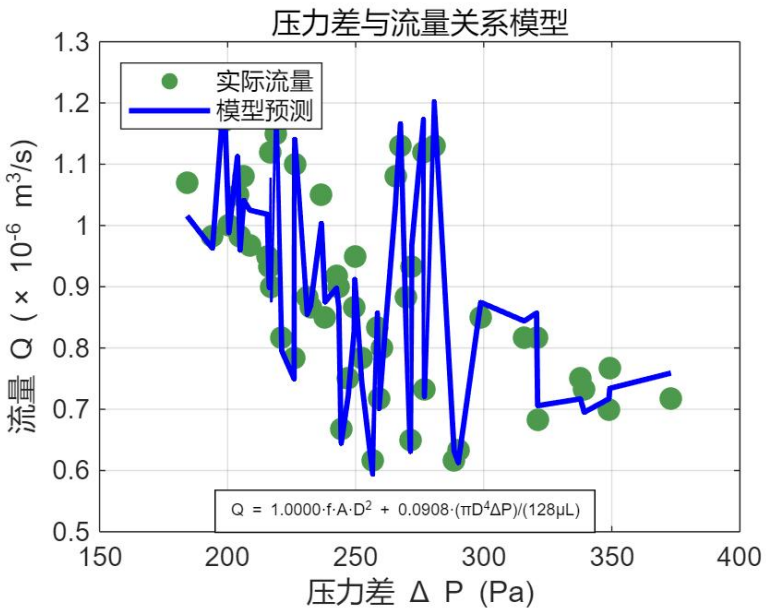



**图 5 压力差-流量关系模型**

综上，成功建立压降-流量解析式模型

$$Q=f\cdot A\cdot D^{2}+\frac{0.0908(\pi D^{4}\cdot \Delta p)}{128\mu\cdot L} \tag{8}$$

## 5.2 问题二模型建立与求解

### 5.2.1 数据准备

第一步：数据读取。利用 Python 读取存储附件 1 表格中的数据，并将其转化为 Dataframe 结构，然后定义模型的输入/输出变量。

第二步：数据处理。为消除机器学习算法建模过程中的数据噪声、异常数据和不规则特征，优化特征表述，增强机器学习模型的性能和泛化能力，需要对数据进行“数据清洗”。该过程包括处理缺失值、异常值、重复值等数据质量问题。经检验，本数据集不存在以上数据质量问题。此外，利用 StandardScaler 对特征数据进行归一化处理。 归一化公式为：

$$X_{i}=\frac{X-X_{min}}{X_{max}-X_{min}}$$

式中：Xi 为归一化后的值；X 为样本原始值；Xmin 为样本最小值；Xmax 为样本最大值。

第三步：切分数据集。经过数据增强，共有 50 组数据可供模型训练。采用 train_test_split( )函数进行数据集的切分，其中 80%用作训练集，20%用作测试集。

以下为处理结果，此处仅展示部分且保留四位小数，完整表格见附录。

表 3 部分数据处理结果

| feature0 | feature1 | feature2 | feature3 | feature4 | feature5 | target |
|---|---|---|---|---|---|---|
| 0.2111 | 0.6971 | 0.5786 | 0.4302 | -0.3987 | -0.3585 | 0.5829 |
| 0.6137 | -0.1639 | 0.0485 | -0.7130 | -1.0908 | -0.2598 | 0.3363 |
| | | | ... | | | |
| -2.4348 | -2.4713 | -2.7790 | 0.4302 | 1.3811 | 3.0484 | -2.1292 |
| -0.4216 | -0.4049 | -0.1283 | 0.4302 | 1.3811 | -0.4616 | -1.5399 |

5.2.2 模型建立

机器学习是一种通过计算机算法从数据中学习规律并优化模型性能的技术，其核心在于通过数据驱动的参数调整，使模型能够准确捕捉数据中的内在模式，从而实现对未知数据的可靠预测与决策。本文综合训练了四种比较典型的机器学习算法对正常肾脏输尿管流量数据[2]进行拟合，建立了四种回归模型来描述输尿管流量与生理指标之间的内在关系。

step1 模型构建

构建四种基于机器学习的模型以实现输尿管流量-压降模型的建立，具体建立流程如下。

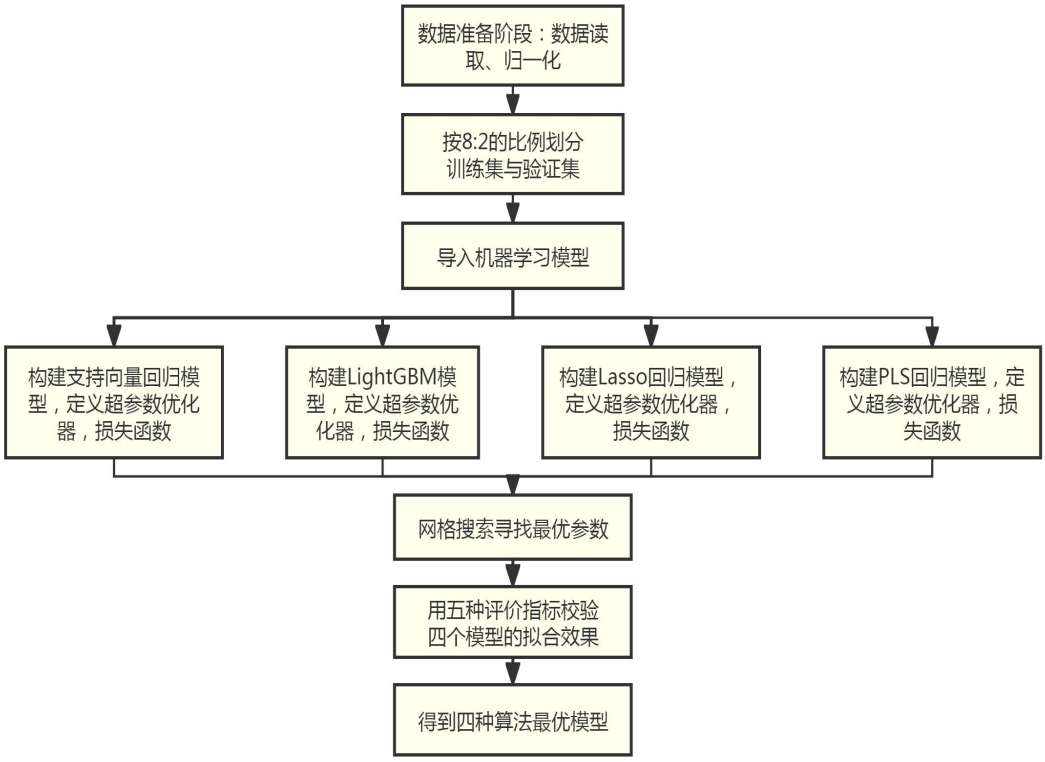



图 6 模型构建流程图

（1）支持向量回归模型：使用 Python 中的 sklearn 库建立支持向量回归模型，并用网格搜索对参数进行调优，经搜索确定核函数为 linear,正则化系数为 1，不敏感带宽为 0.1, 该参数组合下模型拟合效果最好, 测试集评估指标 $R^2$可达 0.97，具体调参范围见下表，测试集模型预测效果见下图

表 4 支持向量回归最优参数表

| 模型 | 参数名称 | 网格搜索范围 | 最优参数值 |
|---|---|---|---|
| 支持向量回归 | kernel | ['linear', 'rbf'] | linear |
| | C | [0,1,5,10] | 1 |
| | epsilon | [0.01,0.1,0.5] | 0.1 |

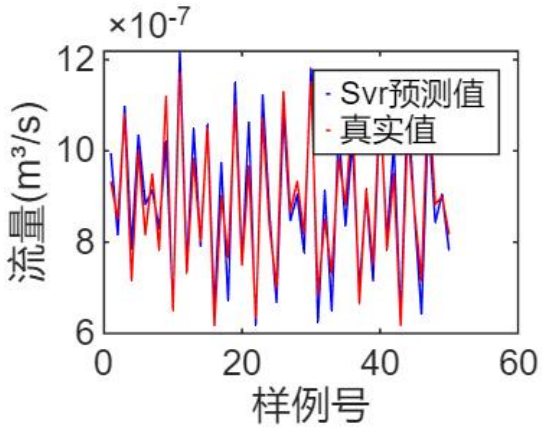



图 7 流量预测图

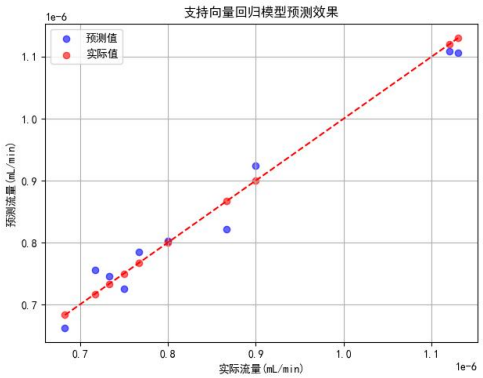



图 8 回归预测模型效果

（2）轻量级梯度提升( LightGBM): 使用 python 的 lightgbm 库建立该回归模型，设置基础参数并采用网格搜索方法对关键超参数（包括学习率、叶子节点数、树深度、子样本数等）进行调优，通过 5 折交叉验证确定最佳参数组合，最优结果见下表；最终模型在测试集上表现出色（$R^2$=0.9562）,具体调参范围见下表，测试集预测效果见下图:

表 5 轻量级梯度提升最优参数表

| 模型 | 参数名称 | 网格搜索范围 | 最优参数值 |
|---|---|---|---|
| LGBM | learning_rate | [0.001,0.05,0.01,0.1] | 0.1 |
| | max_depth | [3,5,7,9,11] | 5 |
| | num_leaves | [15,31,63,100] | 15 |
| | min_child_samples | [5,10,15,20] | 5 |
| | colsample_bytree | [0.6,0.8,1.0] | 0.6 |
| | subsample | [0.5,0.8,1.0] | 0.5 |

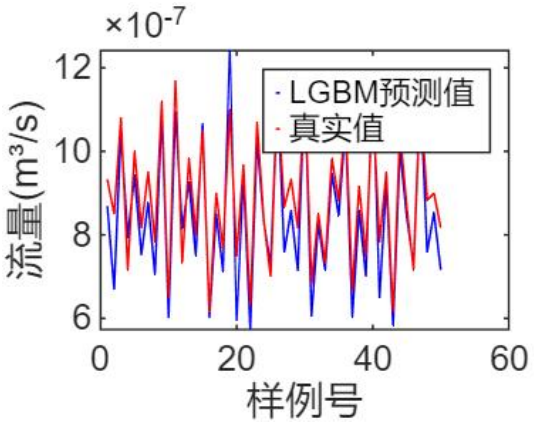



图 9 流量预测图

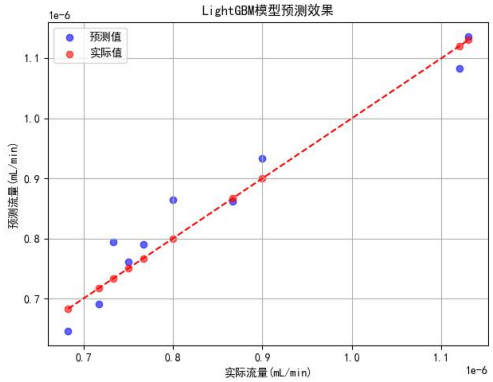



图 10 回归预测模型效果

（3）Lasso 回归模型：使用 python 的 sklearn 库建立 lasso 回归模型，通过网格搜索优化 Lasso 模型的关键参数（包括正则化强度 alpha、最大迭代次数 max_iter 和系数更新方法 selection），采用 5 折交叉验证和负均方误差评估指标确定最优参数组合，模型最终 MAPE 为 1.969%，$R^2$达 0.9562，具体调参数据见下表，拟合效果见下图

表 6 Lasso 回归最优参数表

| 模型 | 参数名称 | 网格搜索范围 | 最优参数值 |
|---|---|---|---|
| Lasso 回归 | alpha | [0.001,0,01,0,1,1] | 0.001 |
| | max_iter | [1000,5000,10000] | 1000 |
| | selection | ['cyclic','random'] | random |

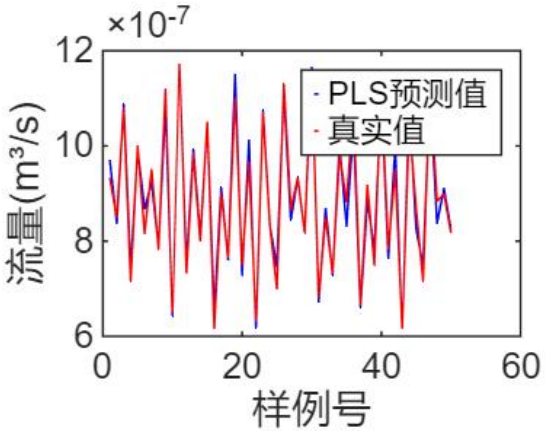



图 11 流量预测图

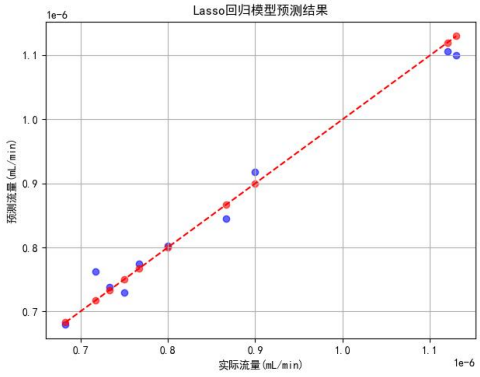



图 12 回归预测模型效果

（4）偏最小二乘回归（PLS）:使用 python 的 sklearn 库建立偏最小二乘回归模型通过网格搜索优化 PLS 模型的关键参数（包括成分数 n_components 和标准化选项 scale），确定最优参数组合，具体数据及调参结果如下：

表 7 偏最小二乘最优参数表

| 模型 | 参数名称 | 网格搜索范围 | 最优参数值 |
|---|---|---|---|
| 偏最小二乘回归 | n_components | [1,2,3,4,5] | 5 |
| | scale | [True,Flase] | Flase |

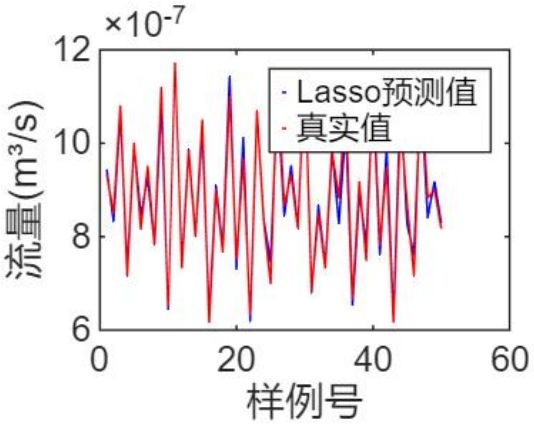



图 13 流量预测图

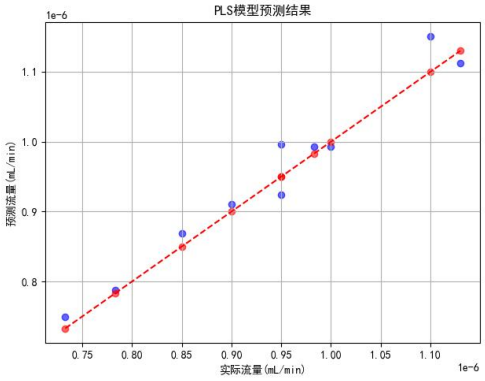



图 14 回归预测模型效果

5.2.3 模型评估分析与结果

表 8 模型评估

| 模型 | MSE | RMSE | MAE | MAPE | $R^2$ |
|---|---|---|---|---|---|
| 支持向量回归 | 6.312e-16 | 2.512e-8 | 2.211e-8 | 2.704% | 0.972 |
| LGBM | 1.316e-15 | 3.627e-8 | 3.038e-8 | 3.801% | 0.943 |
| Lasso 回归 | 4.397e-16 | 2.097e-8 | 1.66e-8 | 1.969% | 0.981 |
| PLS | 6.576e-16 | 2.564e-8 | 2.085e-8 | 2.178% | 0.954 |

（1）均方误差（MSE）反映预测值与真实值差异的平方和。Lasso 回归的 MSE，是四个模型中最小的，说明其整体误差最小；支持向量回归和 PLS 的 MSE 处于相近水平；LGBM 的 MSE 相对较大，整体误差控制不如前三者。

（2）均方根误差（RMSE）是 MSE 的平方根，能更直观反映预测值与真实值的偏差程度。Lasso 回归的 RMSE 最小，表明其预测精度最高；

（3）平均绝对误差（MAE）表示预测值与真实值绝对误差的平均值。Lasso 回归的 MAE 最小，意味着其平均预测偏差最小；PLS、支持向量回归均大于 Lasso 回归；LGBM 的 MAE 是最大的，平均预测偏差相对较大。

（4）平均绝对百分比误差（MAPE）以相对比例形式衡量预测误差。Lasso 回归的 MAPE 最小，说明在相对比例上预测精度最高；PLS 的 MAPE、支持向量回归为 2.704%；LGBM 的 MAPE 最大，预测值的相对误差相对较大。

（5）决定系数（$R^2$）衡量模型对数据的拟合程度。Lasso 回归的为 0.981，是最大的，表明其对数据的拟合效果最好，能解释的因变量变异比例最高。

总体而言，四个模型这五项评估指标的横向对比中，Lasso 回归在 MSE、RMSE、

MAE、MAPE 和 $R^2$上均表现最优，在该预测任务中具有最高的预测精度和最佳的拟合效果。支持向量回归和 PLS 也有较好的表现，而 LGBM 相对来说在各项指标上表现稍逊。

## 5.3 问题三模型验证与优化

### 5.3.1 求解思路

此处假设输尿管流量等同于泌尿量，由于患者只有左肾功能异常，此处仅关心左肾状态。经问题二分析，Lasso 回归模型在各项评价指标中均表现最好，故选用该模型预测结果与解析式模型进行比较，对比手术前后左肾泌尿量预测值与实际数据。

**表 9 左肾泌尿量预测表**

| | 实际值 | 解析式模型 | AI 模型 |
|---|---|---|---|
| 手术前(ml/min) | 1.2100 | 1.2207 | 1.6139 |
| 手术后(ml/min) | 13.1100 | 13.2440 | 17.8694 |

### 5.3.2 模型验证

**表 10 误差分析表**

| 模型 | 手术前 | | 手术后 | |
|---|---|---|---|---|
| | 绝对误差 (ml/min) | 相对误差 | 绝对误差 (ml/min) | 相对误差 |
| 解析式模型 | 0.0107 | 0.88% | 0.134 | 1.02% |
| AI 模型 | 0.4039 | 33.38% | 4.7693 | 36.30% |

误差分析：解析式模型误差较小,在可接受范围之内。误差来源一方面推测是测量输尿管数据时存在系统误差，导致模型建立时数据拟合参数有偏差。另一方面，机理分析时的合理假设忽略了一部分影响因素[3]，可能对最终结果产生影响。即使是手术前也可以预测，体现了灰箱模型的优越性。

AI 模型仍存在一定误差。分析其原因，一方面在于训练数据均为健康人身体数据，病人数据超出训练集范围，因此机器学习算法预测存在较大误差。另一方面在于训练数据过少，导致模型训练不够充分，无法很好地捕捉数据特征，造成术后预测误差。

### 5.3.3 局限性分析与优化建议

解析式模型局限性分析与优化建议：

首先应考虑输尿管弹性及蠕动产生的形变[4]，不仅仅用修正系数 $k_2$描述形变的非线性变化，可以详细描述蠕动导致输尿管边界改变给带来的影响，假设蠕动为正弦波会更贴近现实，还应考虑输尿管可能会拐弯，在拐弯处可能产生湍流，另一方面考虑流体性质，可基于 navier-stokes 方程完整描述流体运动状态，尽可能将灰箱模型转化为白箱模型，增强模型泛用性与可解释性。

其次本模型未考虑输尿管入口及出口状态，不适用于输尿管与肾盂或膀胱连接处，可对这些位置单独讨论建模，提高模型泛用性。

AI 模型局限性分析与优化建议：

针对当前AI模型在测试数据上出现的过拟合及预测效果欠佳问题，经深入分析发现其核心原因在于：

（1）模型训练仅基于健康个体的输尿管流量数据，缺乏病理状态下的样本特征；

（2）现有数据量较少未能充分训练模型；

（3）由于缺少大量数据进行验证，无法保证其具有良好的延展性。

基于上述分析，本研究提出以下关键改进方案：

（1）扩充健康人群的样本数据量，建立更全面的参考数据库；

（2）系统收集术前患者的病理生理数据，特别是不同严重程度的肾积水病例，以更全面精确的对患者状况进行预测。

# 六、模型分析检验

### 6.1.1 模型一基于动态调参的灵敏度分析

本小节基于动态调参方法，依次调节了以下 6 个参数：蠕动频率上升 5%，蠕动幅度上升 5%、管径增加 5%、输尿管长度增加 5%、尿液粘度上升 5%、压降大小上升 5%。通过将这些新的流量数据与原数据计算出的流量进行对比分析，可以清楚观察到模型在不同参数设定下的表现。

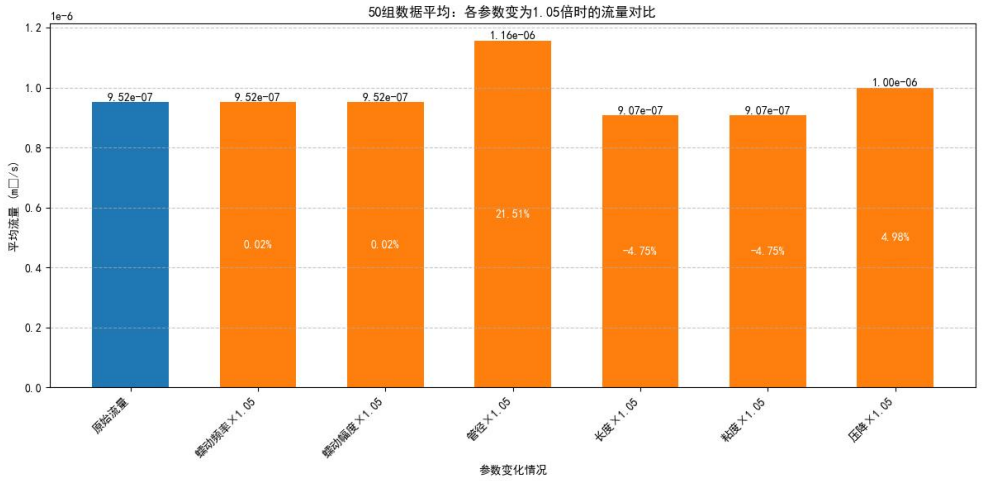



图 15 动态调参的灵敏度分析

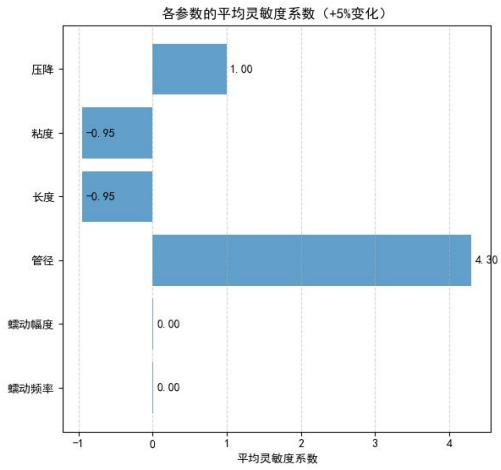



图 16 参数平均灵敏度系数

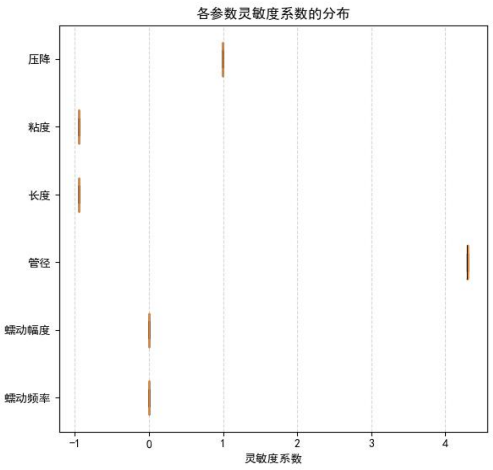



图 17 参数灵敏度分布图

分析结果显示，该模型呈现不均衡的特征，对于管径以外的因素，尽管有所调整，流量数据的波动依然较小。这一发现表明，模型在面对大部分参数变化时能够保持稳定的效果，验证了其稳健性和抗风险能力。

总之，灵敏度分析说明解析式模型具有较强抗风险能力，管径对流量影响最大。

### 6.1.2 模型二特征重要性分析

通过对 Lasso 回归模型训练过程中的绝对值系数参数进行降序排列，量化各特征对模型预测的影响程度。并绘制能反映特征重要性的箱线图，以系数绝对值为横轴、特征名称为纵轴，直观展示特征重要性的相对大小。

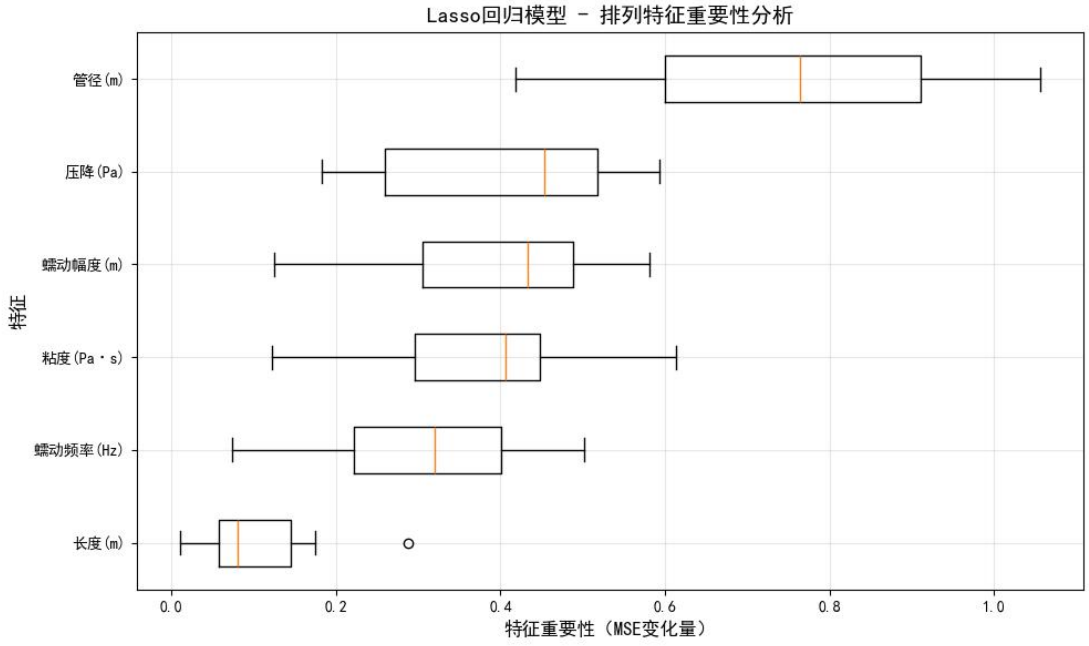



图 18 特征重要性的箱线图

从这张特征重要性箱线图可以看出，各特征对模型的影响程度存在显著差异。“管

径”这一特征对模型训练影响程度最大，箱体最短且中位数接近 0，表示在大多数情况下长度对模型的影响较小，圆圈表示其存在一个异常值，意味着在某些特殊情况下长度可能会对模型产生较大影响，但这种情况较为罕见。整体而言，压降、蠕动幅度、粘度、蠕动频率的箱体长度和中位数依次降低，反映出它们对模型的重要性逐渐减弱。

6.2.1 模型一的误差分析

首先，该数学模型是在合理假设下的理想化状态，在实际中有更多的因素需要考虑。例如，蠕动频率以及蠕动幅度的稳定性、输尿管的材质弹性、管道的入口/出口效应等，这些因素都会对实际流量造成影响，产生误差。

其次，该数学模型是基于机理分析进行数据拟合完成的，由于数据集较少，拟合结果可能存在一定的偶然性，这是无可避免的局限。

6.2.2 模型二的误差分析

首先，训练数据集仅包含健康个体的输尿管流量特征，缺乏病理状态下的特征数据，可能导致模型对异常情况的识别能力不足；其次，现有样本量（仅 50 例）难以充分训练模型的泛化能力；由于验证数据匮乏（仅 1 例），模型在临床应用中的延展性和鲁棒性无法得到有效验证。为此，本研究提出系统性优化方案：一方面将健康人群样本量扩充至 300 例以上，建立更具代表性的正常生理参数基准数据库；另一方面重点采集 200 例以上涵盖不同分型（轻度、中度和重度）的肾积水患者术前术后数据，以提升模型对疾病状态的识别精度和临床预测价值，从而构建一个更加全面、可靠的回归模型。

# 七、模型评价与推广

本研究为临床评估手术效果提供了量化工具，并为 AI 在泌尿动力学中的应用探索了新路径。未来可通过多模态数据融合与实时反馈机制，进一步优化模型精度及手术方案的个性化设计。

7.1.1 数学模型的优点

（1）受外界波动因素影响较小，具有较强的稳健性和抗风险能力；

（2）基于灰箱模型，可通过健康病人的样本较为准确地预测病人的生理情况；

（3）可解释性强：机理与数据共同构建模型，即有一定可解释性又可避免对复杂耦合的分析；

（4）对数据量依赖低:数学模型可基于少量数据,利用物理定律直接构建模型。在数据量有限的情况下，也能通过理论推导保证模型的合理性，无需大规模数据支撑。

7.1.2 数学模型的不足

当前假设中尚未涵盖部分关键影响因素，例如蠕动频率与蠕动幅度的稳定性差异、输尿管自身的材质弹性特性，以及管道入口/出口处的流动效应等。这些未被纳入的因素在实际场景中会通过改变流体运动状态、管道形变规律或边界条件，对流量产生不可忽视的干扰，进而导致模型预测结果与真实值之间出现偏差。

7.2.1 AI 模型的优点

（1）适用于高维数据场景：当数据量较大时，机器学习能够通过算法自动筛选

数据集中的关键特征、降低冗余信息，捕捉特征间的高阶交互关系，提高数据处理和利用效率；

（2）处理复杂非线性问题能力更强：机器学习无需预先假设数据间的数学关系，能通过算法自动捕捉数据中的复杂非线性模式 ；

### 7.2.2 AI 模型的不足

（1）缺乏已有知识整合能力：机器学习主要依赖数据驱动，若领域内存在明确的物理规律或数学定理，难以直接融入模型；

（2）依赖大量数据：机器学习需要足够的数据才能训练出稳定的模型，当数据不足时容易出现过拟合或欠拟合的情况；

（3）可解释性较弱：通过机器学习构建的算法模型难以用直观的数学逻辑解释预测结果。

### 7.2.3 小结

综上，数学模型更适合问题逻辑清晰、有成熟理论支撑的场景，其核心优势在于解释性和理论严谨性；而机器学习更擅长处理数据驱动、规律复杂、高维动态的问题，核心优势在于灵活性和对隐性规律的挖掘能力。在实际应用中常结合二者以兼顾性能与可解释性。

## 7.3 模型的推广

本文建模方法可推广至泌尿系统其他疾病的流体动力学研究领域，尤其适用于输尿管结石患者的尿流率预测，通过量化分析结石梗阻对尿流动力学参数的影响，为临床诊断和治疗决策提供客观依据。

## 参考文献


[1] 张波.《A novel suture-free nephroplication with 4D-printed iodegradable bag for giant hydronephrosis: initial experience from hydromechanics and clinical application》[J].《International Journal of Surgery》, 111(2):p 2304-2308,2025 年 2 月

[2] 侯钊，双卫兵.《机器学习算法的概述及其在生物医学中的应用》[J].《中国医学工程》,33(3):p 72-78,2025 年 3 月

[3] 柳明明，王黉，汪哲丞，陈丹..《基于机器学习构建高血压肾病预测模型研究》[J].《中国现代医生》,63(15):p 7-10

[4] 秦琪琪,《不同程度肾积水对肾盂肾盏流体力学变化影响的模型研究》[D].山西：山西医科大学.2020

# 附录

## 附录 1 支撑材料

| 文件名列表 |
| --- |
| **问题一相关代码** |
| fit_values_of_k1k2.m |
| **问题二相关文件** |
| svr_prediction_picture.m |
| LGBM_prediction_picture.m |
| Lasso_prediction_picture.m |
| PLS_prediction_picture.m |
| best_lasso_model.pkl |
| best_svr_model.pkl |
| best_pls_model.pkl |
| lgbm_flow_model.pkl |
| Lasso_model.py |
| LGBM_model.py |
| PLS_model.py |
| svr_model.py |
| 测试数据.xlsx |
| 机器学习预测结果.xlsx |
| scaled_data.xlsx |
| **其余部分相关代码** |
| 模型一灵敏度分析.py |

## 附录 2 问题一相关代码

MATLAB

约束最小二乘求解 $K_1$，$K_2$最佳值

```
data = readtable("C:\Users\29813\Desktop\附件 1 (1).xlsx", 'Sheet', '

正常肾脏输尿管流量数据');

f = data.f_Hz_;
A = data.A_m_;
D = data.D_m_;
L = data.L_m_;
u = data.u_Pa_s_;
Dp = data.Dp_Pa_;
Q_actual = data.Q_m__s_;
```

```
% 计算模型中的两项

term1 = f .* A .* (D.^2); % 蠕动项

term2 = (pi * D.^4 .* Dp) ./ (128 .* u .* L); % 压力差项


% 构建线性系统：Q_actual = k1*term1 + k2*term2

X = [term1, term2]; % 设计矩阵


% 使用约束最小二乘法求解 k1 和 k2 (k1>0)

lb = [0, -Inf]; % 下界约束：k1 >= 0

ub = [Inf, Inf]; % 无上界约束


% 设置优化选项

options = optimoptions('lsqlin', 'Display', 'off', 'Algorithm',
  'interior-point');


% 求解约束最小二乘问题

params = lsqlin(X, Q_actual, [], [], [], [], lb, ub, [], options);
k1 = params(1);
k2 = params(2);


% 计算模型预测的流量

Q_pred = k1*term1 + k2*term2;


% 计算模型性能指标

R2 = 1 - sum((Q_actual - Q_pred).^2) / sum((Q_actual -
mean(Q_actual)).^2);
RMSE = sqrt(mean((Q_actual - Q_pred).^2));


% 可视化结果

figure('Position', [100, 100, 1200, 500])
```

```
% 实际流量对比预测流量
subplot(1,2,1)
scatter(Q_actual*1e6, Q_pred*1e6, 40, 'filled', 'MarkerFaceColor',
[0.2 0.4 0.8])
hold on
plot([0 max(Q_actual)*1e6], [0 max(Q_actual)*1e6], 'r--',
'LineWidth', 1.5)
hold off
xlabel('实际流量 Q_{actual} (\times 10^{-6} m^3/s)')
ylabel('预测流量 Q_{pred} (\times 10^{-6} m^3/s)')
title(sprintf('流量预测效果 (R^2 = %.4f, RMSE = %.2e)', R2, RMSE))
grid on
axis equal
box on
legend('数据点', '理想拟合线', 'Location', 'northwest')

% 残差分析
subplot(1,2,2)
residuals = Q_actual - Q_pred;
scatter(Q_pred*1e6, residuals*1e6, 40, 'filled', 'MarkerFaceColor',
[0.8 0.4 0.2])
hold on
yline(0, 'r--', 'LineWidth', 1.5)
hold off
xlabel('预测流量 Q_{pred} (\times 10^{-6} m^3/s)')
ylabel('残差 (\times 10^{-6} m^3/s)')
title('残差分析图')
grid on
box on

% 显示参数结果
fprintf('最佳参数值: k1 = %.6f (大于 0), k2 = %.6f (大于 0)\n', k1, k2);
fprintf('模型性能: R² = %.4f, RMSE = %.2e m³/s\n', R2, RMSE);
```

```
% 分析压力差 Dp 与流量的关系
figure('Position', [100, 100, 800, 600])
scatter(Dp, Q_actual*1e6, 70, 'filled', 'MarkerFaceColor', [0.3 0.6
0.3])
hold on

% 按 Dp 排序用于绘制拟合曲线
[sorted_Dp, idx] = sort(Dp);
Q_pred_sorted = Q_pred(idx);
plot(sorted_Dp, Q_pred_sorted*1e6, 'b-', 'LineWidth', 2)

xlabel('压力差 \Delta P (Pa)')

ylabel('流量 Q (\times 10^{-6} m^3/s)')

title('压力差与流量关系模型')

legend('实际流量', '模型预测', 'Location', 'northwest')

grid on
box on

% 添加模型方程标注
equation_text = sprintf('Q = %.4f·f·A·D^2 + %.4f·(πD^4ΔP)/(128μL)',
k1, k2);
annotation('textbox', [0.25, 0.15, 0.5, 0.05], 'String',
equation_text, ...
'FitBoxToText', 'on', 'BackgroundColor', 'white', ...
'FontSize', 6, 'HorizontalAlignment', 'center');
```

## 附录 3 问题二相关代码

PYTHON

（1）数据预处理

```
df = pd.read_excel("附件 1 .xlsx", sheet_name="正常肾脏输尿管流量数据
")
df.fillna(0, inplace=True)
df = df.head(60)
```

```
# 清洗列名
df.columns = [re.sub(r'[\/\\:：  \s\.]', '_', col) for col in df.columns]
# 选择特征和目标
X = df.iloc[:, 1:7].values  # 前 6 列作为特征
y = df.iloc[:, -1].values  # 最后一列作为目标
# 数据标准化
scaler_X = StandardScaler()
scaler_y = StandardScaler()
X_scaled = scaler_X.fit_transform(X)
y_scaled = scaler_y.fit_transform(y.reshape(-1, 1)).ravel()
joblib.dump(scaler_X,"scaler_X.pkl")
joblib.dump(scaler_y,"scaler_y.pkl")
# 划分数据集
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_scaled, test_size=0.2, random_state=42)
```

（2）模型训练

Lasso 模型：

```
# 使用网格搜索寻找最优参数
param_grid = {
    'alpha': [0.001, 0.01, 0.1, 1, 10, 100],  # 正则化强度
    'max_iter': [1000, 5000, 10000],  # 最大迭代次数
    'selection': ['cyclic', 'random']  # 系数更新方法
}
lasso = GridSearchCV(
    Lasso(),
    param_grid,
    cv=5,
    scoring='neg_mean_squared_error',
    n_jobs=-1
)
```

```
lasso.fit(X_train, y_train)
best_lasso = lasso.best_estimator_
# 预测测试集
y_pred_scaled = best_lasso.predict(X_test)
# 反标准化
y_test_orig = scaler_y.inverse_transform(y_test.reshape(-1, 1)).ravel()
y_pred_orig = scaler_y.inverse_transform(y_pred_scaled.reshape(-1,
1)).ravel()
joblib.dump(best_lasso,'best_lasso_models.pkl')
```

svr 模型：

```
    # 使用网格搜索寻找最优参数
param_grid = {
    'kernel': ['rbf', 'linear'],
    'C': [0.1, 1,5,10],
    'epsilon': [0.01, 0.1, 0.5]
}

svr = GridSearchCV(
    SVR(),
    param_grid,
    cv=5,
    scoring='neg_mean_squared_error',
    n_jobs=-1
)
svr.fit(X_train, y_train)

# 获取最佳模型
best_svr = svr.best_estimator_
# 预测测试集
y_pred_scaled = best_svr.predict(X_test)

# 反标准化
y_test_orig = scaler_y.inverse_transform(y_test.reshape(-1, 1)).ravel()
y_pred_orig = scaler_y.inverse_transform(y_pred_scaled.reshape(-1,
1)).ravel()
```

LGBM 模型

```
    params = {
    'boosting_type': 'gbdt',
    'objective': 'regression',
    'metric': 'rmse',
    'verbose': -1,
    'random_state': 42,
    'n_jobs': -1
}

# 参数搜索空间

param_grid = {
    'learning_rate': [0.001,0.01, 0.05, 0.1],
    'num_leaves': [15, 31, 63,100],
    'max_depth': [3, 5, 7,9,11],
    'min_child_samples': [5, 10, 15,20],
    'subsample': [0.5,0.8, 1.0],
    'colsample_bytree': [0.6,0.8, 1.0]
}

# 网格搜索

lgb_model = lgb.LGBMRegressor(**params)
grid_search = GridSearchCV(
    estimator=lgb_model,
    param_grid=param_grid,
    cv=5,
    scoring='neg_mean_squared_error',
    n_jobs=-1,
    verbose=1
)

print("开始参数搜索...")

grid_search.fit(X_train, y_train)
best_lgb = grid_search.best_estimator_

print("\n 最佳参数组合:", grid_search.best_params_)


model_filename = f'lgbm_flow_model.pkl'
joblib.dump(best_lgb, model_filename)

print("模型已导出")
```

```
best_lgb = joblib.load('lgbm_flow_models.pkl')
# 预测测试集
y_pred_scaled = best_lgb.predict(X_test)
# 反标准化
y_test_orig = scaler_y.inverse_transform(y_test.reshape(-1, 1)).ravel()
y_pred_orig = scaler_y.inverse_transform(y_pred_scaled.reshape(-1,
1)).ravel()
```

PLS 模型

```
    # 使用网格搜索寻找最优成分数
param_grid = {
    'n_components': range(1, 6),
    'scale': [True, False]
}

pls = GridSearchCV(
    PLSRegression(),
    param_grid,
    cv=5,
    scoring='neg_mean_squared_error',
    n_jobs=-1
)

print("开始训练 PLS 模型...")
pls.fit(X_train, y_train.ravel())

# 获取最佳模型
best_pls = pls.best_estimator_
print(f"\n 最佳参数: {pls.best_params_}")
joblib.dump(best_pls,'best_pls_models.pkl')

# 预测测试集
y_pred_scaled = best_pls.predict(X_test)
```

```
# 反标准化
y_test_orig = scaler_y.inverse_transform(y_test)
y_pred_orig = scaler_y.inverse_transform(y_pred_scaled.reshape(-1, 1))
```

（3）模型评估

```
def calculate_metrics(y_true, y_pred):
    """计算并返回所有评估指标"""
    metrics = {}
    # 基础指标
    metrics['MSE'] = mean_squared_error(y_true, y_pred)
    metrics['RMSE'] = np.sqrt(metrics['MSE'])
    metrics['MAE'] = mean_absolute_error(y_true, y_pred)
    metrics['R2'] = r2_score(y_true, y_pred)
    epsilon = 1e-10  # 极小值避免除零
    mape = np.mean(np.abs((y_true - y_pred) / (y_true + epsilon)) * 100)
    metrics['MAPE(%)'] = mape

    return metrics

# 计算所有指标
metrics = calculate_metrics(y_test_orig, y_pred_orig)
# 打印评估结果
print("\n===  模型评估结果  ===")
print(f"最佳参数: {lasso.best_params_}")
print("{:<10} {:<15}".format('指标', '值'))
print("-" * 25)
for name, value in metrics.items():
    if name == 'MAPE(%)':
        print(f"{name:<10} {value:}%")
    else:
        print(f"{name:<10} {value:}")
```

（4）可视化预测结果

```
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)

plt.scatter(y_test_orig, y_pred_orig, alpha=0.6, c='blue', label='预测值')

plt.scatter(y_test_orig, y_test_orig, alpha=0.6, c='red', label='实际值')

plt.plot([min(y_test_orig), max(y_test_orig)],
         [min(y_test_orig), max(y_test_orig)], 'r--')

plt.xlabel('实际流量(mL/min)')

plt.ylabel('预测流量(mL/min)')

plt.title('Lasso 回归模型预测结果')

plt.grid(True)
plt.tight_layout()
plt.show()
```

MATLAB

绘制机器学习预测实际对比图

```
data=readtable("C:\Users\29813\Desktop\机器学习预测结果.xlsx");

DP=data.Dp;
num=data.x__ID;
plot(num,data.Svr,'b');
hold on;
plot(num,data.Q_m__s_,'r');

xlabel("样例号");

ylabel("流量(m³/s)");

h=legend('Svr 预测值','真实值');

h.ItemTokenSize=[1,1];

plot(num,data.LGBM,'b');
hold on;
plot(num,data.Q_m__s_,'r');
```

```
xlabel("样例号");

ylabel("流量(m³/s)");

h=legend('LGBM 预测值','真实值');
h.ItemTokenSize=[1,1];

plot(num,data.PLS,'b');
hold on;
plot(num,data.Q_m__s_,'r');
xlabel("样例号");

ylabel("流量(m³/s)");

h=legend('PLS 预测值','真实值');
h.ItemTokenSize=[1,1];

plot(num,data.Lasso,'b');
hold on;
plot(num,data.Q_m__s_,'r');
xlabel("样例号");

ylabel("流量(m³/s)");

h=legend('Lasso 预测值','真实值');
h.ItemTokenSize=[1,1];
```

附录 4 机器学习数据标准化后完整数据表格

| feature_0 | feature_1 | feature_2 | feature_3 | feature_4 | feature_5 | target |
|---|---|---|---|---|---|---|
| 0.2111 | 0.6971 | 0.5786 | 0.4302 | -0.3987 | -0.3585 | 0.5829 |
| 0.6137 | -0.1639 | 0.0485 | -0.7130 | -1.0908 | -0.2598 | 0.3363 |
| -0.1916 | 1.1448 | 0.9320 | 1.1924 | 0.1946 | -0.3704 | 1.0197 |
| 0.8725 | 0.1116 | -0.1283 | -1.4751 | -1.3874 | -0.3073 | -0.0588 |
| -0.7668 | 1.3859 | 0.7553 | 0.0492 | -0.1021 | -0.3773 | 0.7820 |
| 1.0738 | -0.4738 | 0.3135 | -0.3319 | -0.7942 | -0.3527 | 0.2383 |

| | | | | | | |
|---|---|---|---|---|---|---|
| -0.4792 | 0.9037 | 0.5786 | 0.8113 | 0.5901 | -0.3186 | 0.6334 |
| 0.3836 | 0.0428 | 0.1368 | -1.0940 | -0.9919 | -0.3469 | 0.1372 |
| 0.7862 | 0.4560 | 0.8437 | 1.5734 | 0.8867 | -0.2864 | 1.1385 |
| -0.0765 | -0.3361 | -0.3050 | -1.8562 | -1.4863 | -0.2926 | -0.2579 |
| 1.2464 | 1.2481 | 1.0204 | 0.4302 | -0.2998 | -0.3795 | 1.2871 |
| -0.3641 | 0.2150 | -0.0399 | -0.7130 | -0.8931 | -0.2860 | -0.0113 |
| 0.4986 | 0.8004 | 0.6670 | -0.3319 | -0.5964 | -0.3850 | 0.7315 |
| 0.0960 | -0.1294 | 0.2252 | 0.0492 | -0.1021 | -0.3058 | 0.1878 |
| 0.9588 | 1.0415 | 0.7553 | 1.1924 | 0.3923 | -0.3342 | 0.9305 |
| -0.6517 | -0.5771 | -0.2166 | -1.0940 | -1.2886 | -0.3102 | -0.3560 |
| 1.3614 | 0.5594 | 0.4019 | -0.7130 | -0.6953 | -0.3577 | 0.4849 |
| -0.2491 | 0.0083 | 0.0485 | 0.8113 | 0.7878 | -0.1995 | 0.0897 |
| 0.6712 | 1.4892 | 1.1088 | 1.9545 | 1.1833 | -0.3466 | 1.0791 |
| 0.2111 | -0.2327 | -0.0399 | -1.4751 | -1.1897 | -0.3217 | 0.0392 |
| 1.0738 | 0.8004 | 0.6670 | -0.3319 | -0.4976 | -0.3676 | 0.6839 |
| -0.4792 | -0.4738 | -0.3933 | -1.8562 | -1.5852 | -0.2702 | -0.3084 |
| 0.3836 | 1.1448 | 0.9320 | 0.4302 | -0.0032 | -0.3968 | 0.9899 |
| 0.7862 | 0.1805 | 0.1368 | -1.0940 | -0.9919 | -0.3080 | 0.2858 |
| -0.0765 | -0.3361 | -0.1283 | 0.0492 | -0.2998 | -0.1999 | -0.1094 |
| 1.2464 | 0.9037 | 0.8437 | 1.5734 | 0.5901 | -0.2973 | 1.1682 |
| -0.3641 | -0.1294 | 0.3135 | -0.7130 | -0.7942 | -0.3392 | 0.3868 |
| 0.4986 | 0.4560 | 0.5786 | 0.8113 | 0.8867 | -0.2919 | 0.5829 |
| 0.0960 | 0.0428 | 0.0485 | -0.3319 | -0.5964 | -0.2395 | 0.2383 |
| 0.9588 | 1.2481 | 1.0204 | 1.1924 | 0.1946 | -0.3554 | 1.2276 |
| -0.6517 | -0.6804 | -0.3050 | -1.4751 | -1.3874 | -0.2332 | -0.1599 |
| 1.3614 | 0.1116 | 0.2252 | -1.0940 | -1.0908 | -0.3324 | 0.3363 |
| -0.2491 | -0.2327 | -0.0399 | 0.4302 | 0.3923 | -0.2115 | -0.0113 |
| 0.6712 | 0.8004 | 0.6670 | 0.0492 | -0.3987 | -0.3721 | 0.7315 |
| 0.2111 | 0.2150 | 0.1368 | -0.7130 | -0.8931 | -0.2947 | 0.4344 |
| 1.0738 | 1.0415 | 0.7553 | 1.5734 | 0.9856 | -0.2996 | 1.0197 |
| -0.4792 | -0.3361 | -0.2166 | -1.8562 | -1.6841 | -0.3249 | -0.2074 |
| 0.3836 | 0.5594 | 0.4019 | -0.3319 | -0.2009 | -0.3271 | 0.5354 |
| 0.7862 | 0.0083 | 0.0485 | 0.8113 | 0.6889 | -0.2132 | 0.0392 |
| -0.0765 | 1.3859 | 1.1088 | 1.9545 | 1.3811 | -0.3578 | 1.1385 |

| | | | | | | |
|---|---|---|---|---|---|---|
| 1.2464 | -0.2327 | -0.0399 | -1.4751 | -1.2886 | -0.3153 | 0.1372 |
| -0.3641 | 0.8004 | 0.6670 | 0.0492 | -0.4976 | -0.3594 | 0.6334 |
| 0.4986 | -0.4738 | -0.3933 | -2.2372 | -1.4863 | -0.2722 | -0.3560 |
| 0.0960 | 1.1448 | 0.9320 | 0.4302 | 0.0957 | -0.3733 | 0.9305 |
| 0.9588 | 0.1805 | 0.1368 | -1.0940 | -0.9919 | -0.3188 | 0.3868 |
| -0.6517 | -0.3361 | -0.1283 | 0.0492 | -0.2009 | -0.1710 | -0.0588 |
| 1.3614 | 0.9037 | 0.8437 | 1.5734 | 0.7878 | -0.2815 | 1.1682 |
| -0.2491 | -0.1294 | 0.3135 | -0.7130 | -0.6953 | -0.3407 | 0.4344 |
| 0.6712 | 0.4560 | 0.5786 | 0.8113 | 0.8867 | -0.3260 | 0.4849 |

**附录 5 特征重要性分析结果**

| 特征名（按重要性降序排列） | 特征重要性数值 |
|---|---|
| 管径（m） | 0.7633 |
| 压降（Pa） | 0.4099 |
| 蠕动幅度（m） | 0.3941 |
| 粘度（Pa.s） | 0.3865 |
| 蠕动频率（Hz） | 0.3139 |
| 长度（m） | 0.0973 |

**附录 6 问题三相关代码**

**AI 模型预测代码：**

```
import joblib
from datetime import datetime
import pandas as pd
import re
# 加载标准化模型
try:
    scaler_X = joblib.load('scaler_X.pkl')  # 特征标准化器
    scaler_y = joblib.load('scaler_Y.pkl')  # 目标变量标准化器
    print("标准化模型加载成功")
except Exception as e:
    print(f"加载标准化模型失败: {str(e)}")
    exit()
```

```
# 加载测试数据

test_df = pd.read_excel("测试数据.xlsx")

test_df.columns = [re.sub(r'[\/\\:：\s\.]', '_', col) for col in test_df.columns]

required_features = test_df.columns[1:7]
print(required_features)
# 确保特征列与训练时一致

X_new = test_df[required_features]

# 4. 标准化特征数据

X_new_scaled = scaler_X.transform(X_new)

# 5. 加载模型并预测

model_path = "best_lasso_model.pkl"
# model_path = "best_pls_model.pkl"
# model_path = "best_svr_model.pkl"
# model_path = "lgbm_flow_model.pkl"

try:
    model = joblib.load(model_path)
    print(f"模型 {model_path} 加载成功")
except Exception as e:
    print(f"模型加载失败: {str(e)}")
    exit()

# 进行预测（标准化后的预测值）

y_pred_scaled = model.predict(X_new_scaled)
# 6. 反标准化预测结果

y_pred = scaler_y.inverse_transform(y_pred_scaled.reshape(-1, 1)).flatten()
y = y_pred*6*10**7
print(y)
# 7. 创建结果 DataFrame

results = pd.DataFrame({
    '样本 ID': test_df.index + 1,
```

```
        '预测流量(mL/min)': y_pred
})
# 8. 保存结果
results_filename = f'流量预测结果
_{datetime.now().strftime("%Y%m%d%H%M")}.xlsx'
results.to_excel(results_filename, index=False)
print(f"\n 预测结果已保存到: {results_filename}")
```

**附录 7 问题三 AI 模型预测结果**

| 模型 | 左肾泌尿量预测结果（ml/min） | | 相对误差 | |
|---|---|---|---|---|
| | 手术前 | 手术后 | 手术前 | 手术后 |
| Lasso | 1.6139 | 17.8694 | 0.3338 | 0.3630 |
| svr | 1.4018 | 13.2977 | 0.1585 | 0.0143 |
| pls | 1.5704 | 17.5334 | 0.2979 | 0.3374 |
| LGBM | 1.8575 | 13.0671 | 0.5351 | -0.0033 |
| 均值 | 1.6109 | 15.4419 | 0.3313 | 0.1779 |
| 真实值 | 1.2100 | 13.1100 | / | / |

**附录 8 问题一灵敏度分析相关代码**

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 设置中文字体
plt.rcParams["font.family"] = ["SimHei", "WenQuanYi Micro Hei", "Heiti TC"]
plt.rcParams['axes.unicode_minus'] = False


# 流量计算公式
def calculate_flow(f, A, D, L, u, P):
    """计算流量 Q = A·f·D² + (0.098·π·D⁴·P)/(128·u·L)"""
    return (np.array(A) * np.array(f) * np.array(D) ** 2 +
            (0.098 * np.pi * (np.array(D) ** 4) * np.array(P)) / (128 * np.array(u) * np.array(L)))
```

```
# 读取数据
file1 = '附件 1.xlsx'
data = pd.read_excel(file1, sheet_name='正常肾脏输尿管流量数据')


# 计算每个参数的灵敏度系数
def calculate_sensitivity(data, param_name, param_factor=1.05):
    """
    计算特定参数的灵敏度系数

    参数:
    - data: 包含所有参数的 DataFrame
    - param_name: 参数名称（DataFrame 列名）
    - param_factor: 参数变化因子，默认为 1.05（+5%）

    返回:
    - sensitivity: 该参数的灵敏度系数数组
    """
    sensitivity = []

    for i in range(len(data)):
        # 提取基准参数值
        base_f = data['蠕动频率(Hz)'].iloc[i]
        base_A = data['蠕动幅度(m)'].iloc[i]
        base_D = data['管径(m)'].iloc[i]
        base_L = data['长度(m)'].iloc[i]
        base_u = data['粘度(Pa·s)'].iloc[i]
        base_P = data['压降(Pa)'].iloc[i]

        # 计算基准流量
        base_flow = calculate_flow(base_f, base_A, base_D, base_L, base_u, base_P)

        # 修改目标参数
        if param_name == '蠕动频率(Hz)':
            modified_f = base_f * param_factor
            modified_flow = calculate_flow(modified_f, base_A, base_D, base_L, base_u, base_P)
        elif param_name == '蠕动幅度(m)':
            modified_A = base_A * param_factor
            modified_flow = calculate_flow(base_f, modified_A, base_D, base_L, base_u, base_P)
        elif param_name == '管径(m)':
            modified_D = base_D * param_factor
            modified_flow = calculate_flow(base_f, base_A, modified_D, base_L, base_u, base_P)
        elif param_name == '长度(m)':
            modified_L = base_L * param_factor
```

```
            modified_flow = calculate_flow(base_f, base_A, base_D, modified_L, base_u, base_P)
        elif param_name == '粘度(Pa·s)':
            modified_u = base_u * param_factor
            modified_flow = calculate_flow(base_f, base_A, base_D, base_L, modified_u, base_P)
        elif param_name == '压降(Pa)':
            modified_P = base_P * param_factor
            modified_flow = calculate_flow(base_f, base_A, base_D, base_L, base_u, modified_P)
        else:
            raise ValueError(f"未知参数: {param_name}")

        # 计算灵敏度系数: (ΔQ/Q) / (ΔX/X)
        flow_change_ratio = (modified_flow - base_flow) / base_flow
        param_change_ratio = param_factor - 1
        sensitivity_coef = flow_change_ratio / param_change_ratio

        sensitivity.append(sensitivity_coef)

    return np.array(sensitivity)


# 计算各参数的灵敏度系数（±5%变化）
param_names = ['蠕动频率(Hz)', '蠕动幅度(m)', '管径(m)', '长度(m)', '粘度(Pa·s)', '压降(Pa)']
param_labels = ['蠕动频率', '蠕动幅度', '管径', '长度', '粘度', '压降']

# 存储所有参数的灵敏度系数
sensitivity_results = {}

# 计算+5%变化的灵敏度
for param in param_names:
    sensitivity = calculate_sensitivity(data, param, param_factor=1.05)
    sensitivity_results[param] = sensitivity

# 计算平均灵敏度系数
avg_sensitivity = {param: np.mean(sensitivity_results[param]) for param in param_names}

# 打印结果
print("各参数平均灵敏度系数（+5%变化）:")
for param, label in zip(param_names, param_labels):
    print(f"{label}: {avg_sensitivity[param]:.2f}")

# 找出最敏感的参数
most_sensitive_param = max(avg_sensitivity, key=avg_sensitivity.get)
most_sensitive_label = param_labels[param_names.index(most_sensitive_param)]
print(f"\n 最敏感的参数: {most_sensitive_label} ({avg_sensitivity[most_sensitive_param]:.2f})")
```

```
# 可视化灵敏度系数
plt.figure(figsize=(12, 6))

# 柱状图：平均灵敏度系数
plt.subplot(1, 2, 1)
y_pos = np.arange(len(param_labels))
sensitivity_values = [avg_sensitivity[param] for param in param_names]
plt.barh(y_pos, sensitivity_values, align='center', alpha=0.7)
plt.yticks(y_pos, param_labels)
plt.xlabel('平均灵敏度系数')
plt.title('各参数的平均灵敏度系数（+5%变化）')
plt.grid(axis='x', linestyle='--', alpha=0.5)

# 为每个柱子添加数值标签
for i, v in enumerate(sensitivity_values):
    plt.text(v + 0.05, i, f'{v:.2f}', va='center')

# 箱线图：展示灵敏度系数的分布
plt.subplot(1, 2, 2)
sensitivity_data = [sensitivity_results[param] for param in param_names]
plt.boxplot(sensitivity_data, labels=param_labels, vert=False)
plt.xlabel('灵敏度系数')
plt.title('各参数灵敏度系数的分布')
plt.grid(axis='x', linestyle='--', alpha=0.5)

plt.tight_layout()
plt.savefig('灵敏度分析结果.png', dpi=300)
plt.show()

# 保存结果到 Excel
sensitivity_df = pd.DataFrame({
    '参数': param_labels,
    '平均灵敏度系数': [avg_sensitivity[param] for param in param_names],
    '最小灵敏度': [np.min(sensitivity_results[param]) for param in param_names],
    '最大灵敏度': [np.max(sensitivity_results[param]) for param in param_names],
    '标准差': [np.std(sensitivity_results[param]) for param in param_names]
})

sensitivity_df.to_excel('灵敏度系数分析.xlsx', index=False)
```

## 附录 9 特征重要性分析相关代码

```
from sklearn.inspection import permutation_importance
import matplotlib.pyplot as plt
# 获取特征名称（假设 df 是原始 DataFrame）

feature_names = df.columns[1:7]  # 前 6 列作为特征

# 计算排列重要性

result = permutation_importance(
    svr,
    X_test,
    y_test,
    n_repeats=30,
    random_state=42,
    scoring='neg_mean_squared_error'
)

# 按重要性排序

sorted_idx = result.importances_mean.argsort()

# 绘制排列重要性箱线图

plt.figure(figsize=(10, 6))
plt.boxplot(
    result.importances[sorted_idx].T,
    vert=False,
    labels=np.array(feature_names)[sorted_idx]
)
plt.title("svr 回归模型 - 排列特征重要性分析", fontsize=14)

plt.xlabel("特征重要性（MSE 变化量）", fontsize=12)

plt.ylabel("特征", fontsize=12)

plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# 打印重要性数值

print("\n=== 排列重要性 ===")

for i in sorted_idx[::-1]:
```

```
        print(f"{feature_names[i]:<20}  重要性:
{result.importances_mean[i]:.4f} (±{result.importances_std[i]:.4f})")
```